Persian version: ../02-implementation.html

Meeting Assistant
Technical Implementation

Document 2 of 3 · Engineering reference for the current MVP · Project overview · Roadmap · CHANGELOG

This document is the engineering reference for the running system: how data flows from input to Jira, how each component is built, the API surface, UI, error handling, and test suite. For product motivation and vision see Document 1; for future work see Document 3.

Contents

1. System Goal

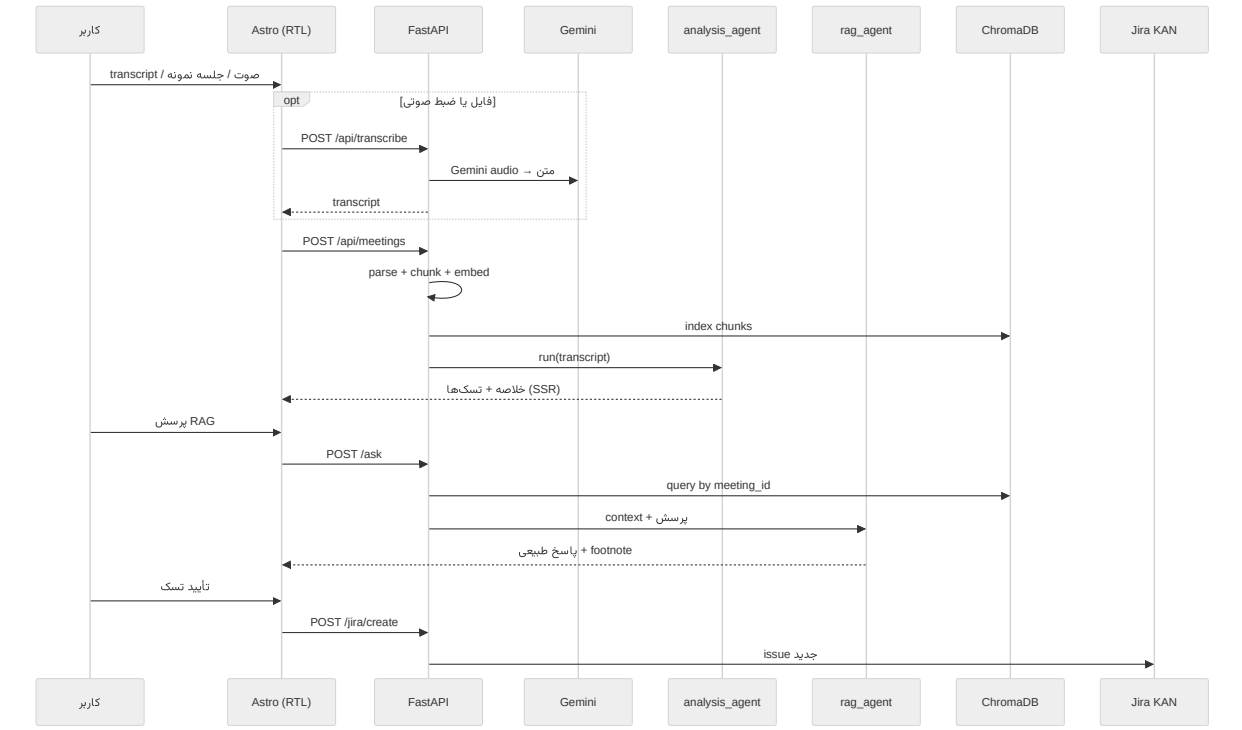
Transform Persian meeting transcripts into operational output: **summary**, **tasks**, **RAG Q&A**, and **Jira issues** — through a pipeline fast enough for live demo.

2. MVP vs Full Vision (YAGNI)

The original concept document described a maximal system. The MVP keeps only what is needed to prove value and explicitly records what is deferred — a disciplined "you ain't gonna need it" stance.

Capability	Full vision (concept doc)	Current MVP
Input	Live recording + streaming STT	Transcript (paste + text file) and **audio** (upload/record ← Gemini)
Analysis	GPT-4 / multiple models	Gemini Flash + Pydantic AI
Search / Vector DB	ElasticSearch	**ChromaDB** embedded (`data/chroma/`)
Task tracking	Jira + Trello + Asana + …	Jira only (`KAN`) — issues in English
RAG	Whole-org archive	Single meeting · multi-meeting planned (Sprint 2)
Calendar / Teams	Full integration	Deferred (next sprint)
Dashboard / notifications	Weekly reports	Cross-meeting task dashboard today · richer reports later

3. End-to-End Data Flow



4. Audio Transcription (Gemini)

Audio input is a **separate step before ingest**, so text paste and sample meeting loading remain unchanged. The same `GOOGLE_API_KEY` and `GEMINI_MODEL` are used — no separate speech-to-text service.

Step	Details
UI	Upload `mp3, wav, m4a, ogg, webm, flac` or **Record** button (MediaRecorder).
API	`POST /api/transcribe` — multipart `file` ← Gemini `generateContent` with inline audio.
Output	Persian transcript in `[MM:SS] name: text` format, editable before analysis.
Limit	Maximum **20 MB** (`MAX_AUDIO_BYTES`).
After transcription	Same path as `POST /api/meetings` — analysis, Chroma, RAG, Jira.

> **Difference from professional STT:** Diarization and live streaming are not part of the MVP; for demo and recorded audio, single-pass Gemini transcription is sufficient.

5. SQLite — Meeting Storage

Item	Value
File	`data/meetings.db`
Table	`meetings` — transcript, summary, key_points, decisions, tasks (JSON)
Migration	Legacy `data/meetings/*.json` files are imported once on first run.

SQLite gives the MVP durable persistence without operations overhead. The schema is deliberately simple so Sprint 5 migration to PostgreSQL (with SQLAlchemy + Alembic) is straightforward.

6. ChromaDB — Vector Store

> **Why Chroma?** The previous version used a hand-rolled JSON store with a Python cosine loop — slow for demo and hard to maintain. ChromaDB is embedded: no separate server, persistence in `data/chroma/`, and fast HNSW search.

Item	Value
Library	`chromadb`
Collection	`meeting_chunks`
Filter	`metadata.meeting_id`

Embedding	`gemini-embedding-001` (from Google API; Chroma only stores/searches)
Distance	cosine (HNSW)

7. Pydantic AI Agents

7.1 analysis_agent

```
from pydantic import BaseModel
from pydantic_ai import Agent

class MeetingAnalysis(BaseModel):
    title: str
    summary: str
    key_points: list[str]
    decisions: list[str]
    tasks: list[MeetingTask]

analysis_agent = Agent(
    'google:gemini-3.5-flash',
    output_type=MeetingAnalysis,
    system_prompt='تحلیل transcript فقط بر اساس متن فارسی.جلسه.',
)
```

7.2 RAG — Smart Behavior

- **Hello / سلام:** No transcript search — a short reply inviting a real question about the meeting.
- **Meeting question:** Chroma top-4 ← internal context for the LLM (raw transcript is never shown in the UI).
- **Answer:** Natural Persian with attribution ("Ali said...") and a short footnote when needed.

```
# Small-talk detection
is_small_talk("hello") -> chitchat_agent, sources=[]
```

```
# Real question
chunks = chroma.query(meeting_id, question)
rag_agent.run(context + question) -> RagAnswer
```

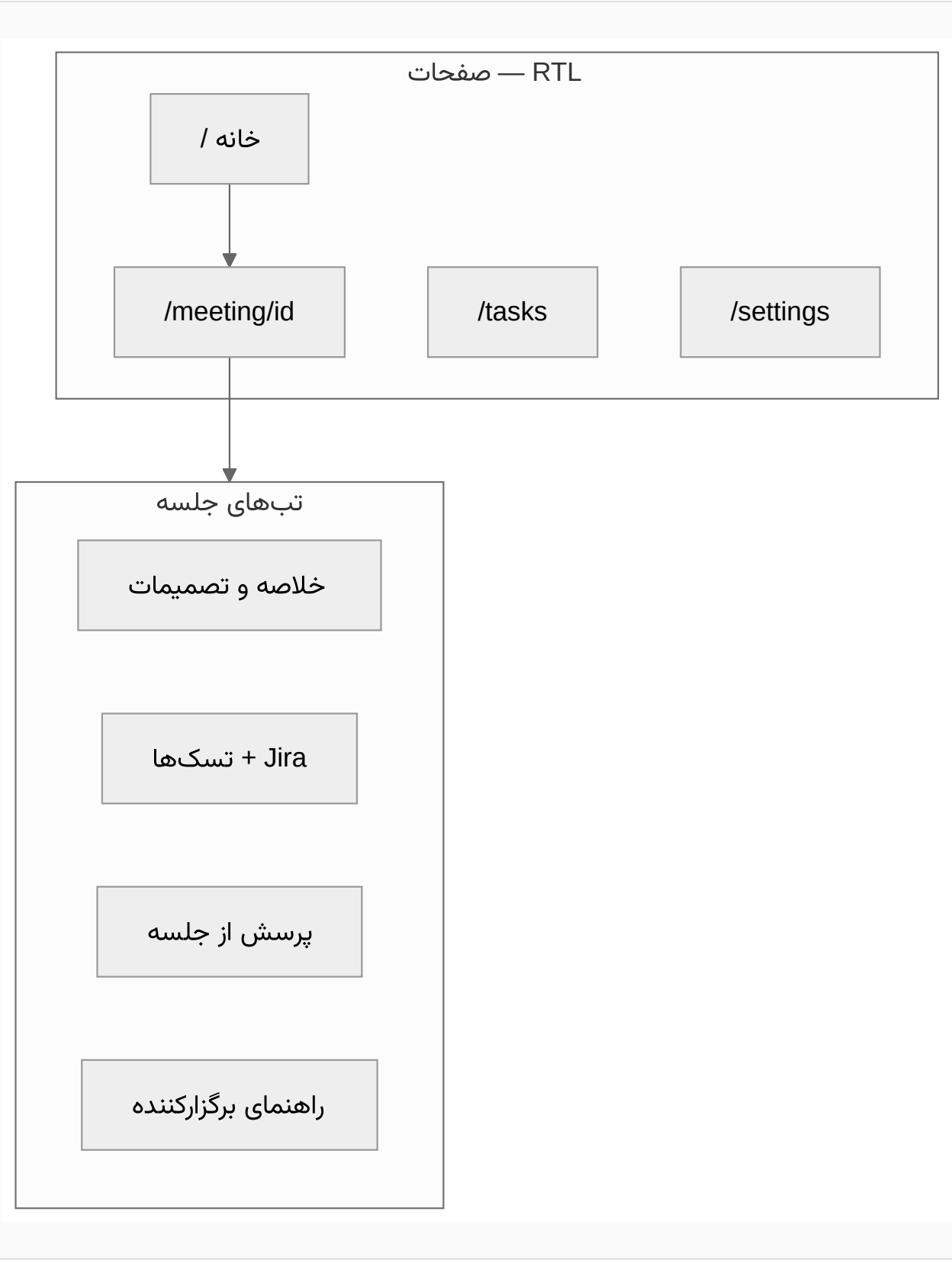
7.3 facilitation_agent

After ingest, the facilitator can view suggestions for improving the next meeting — coaching tone, not blame.

- **Input:** Transcript, `MeetingAnalysis`, speaker participation stats, meeting type.
- **Output:** `FacilitationReport` — positives, improvements, next-meeting agenda, timebox suggestion, coaching summary, and optional facilitator score 1–5.
- **API:** `GET /api/meetings/{id}/facilitation`
- **UI:** "Meeting guide" tab on the meeting page (lazy loaded).

> Jira creation is a direct API route. `summary` comes from `title_en` (Persian fallback); `description` is Persian and structured: **work description** (`detail`), **acceptance criteria** (`acceptance_criteria`), meeting, context, assignee, deadline, priority. The UI remains Persian.

8. User Interface (Astro SSR)



RTL/LTR rules:

- `html lang="fa" dir="rtl"` — Persian UI at `/fa/`.
- `html lang="en" dir="ltr"` — English UI at `/en/` (Inter font); i18n dictionaries in `frontend/src/i18n/`.
- Code, environment variables, URLs, and API names use `dir="ltr"`.

- Mermaid diagrams render inside an LTR wrapper.

The `/meeting/[id]` page fetches data on the **Astro server** from FastAPI (not client-only JS), fixing "stuck on loading" and improving first paint.

Home — meeting archive, search/filter, synthetic meetings, ingest form

Meeting · Summary — key points, decisions, participation stats

Meeting · Tasks — action items, Jira preview and create

Meeting · Ask — RAG with natural answers and sources

Tasks — cross-meeting dashboard with "open only" filter

Settings — API status, speaker←Jira mapping, .env guide

Screenshots are generated with `./scripts/capture-ui-screenshots.sh` (Playwright) — backend and frontend must be running.

9. API Endpoints

Method	Path	Description
GET	/api/health	Service health — Google/Jira/DB status
GET	/api/meetings	List meetings
GET	/api/meetings/{id}	Get one meeting
DELETE	/api/meetings/{id}	Delete meeting
GET	/api/tasks	Cross-meeting action items

GET	/api/meetings/synthetic	List bundled synthetic meetings
POST	/api/meetings/synthetic/{file_id}	Ingest one synthetic meeting
POST	/api/transcribe	Transcribe audio file with Gemini
POST	/api/meetings	Ingest + `analysis_agent`
GET	/api/meetings/{id}/export	Export meeting summary
GET	/api/meetings/{id}/speakers	Speaker participation stats
GET	/api/meetings/{id}/facilitation	Facilitator guide — `facilitation_agent`
POST	/api/meetings/{id}/ask	RAG — `rag_agent`
POST	/api/meetings/{id}/jira/preview	Preview Jira issues
POST	/api/meetings/{id}/jira/create	Create Jira issues
GET / PUT / DELETE	/api/settings/assignee-map	Speaker ← Jira account mapping

10. Synthetic Data

File	Topic
meeting-01-scrum.txt	Weekly standup
meeting-02-planning.txt	MVP planning
meeting-03-review.txt	Sprint review
meeting-04-client-kickoff.txt	Client kickoff — scope, demo, security, pilot

Line format: `[14:02] Ali: speech text`.

11. Environment Configuration (.env)

```
GOOGLE_API_KEY=...
EMBEDDING_MODEL=gemini-embedding-001
GEMINI_MODEL=google:gemini-3.5-flash
MAX_AUDIO_BYTES=20971520
JIRA_SITE_URL=https://your-site.atlassian.net
JIRA_PROJECT_KEY=KAN
JIRA_EMAIL=your@email.com
JIRA_API_TOKEN=...
PUBLIC_BACKEND_URL=http://127.0.0.1:8000
BACKEND_PORT=8000
```

12. Project Structure

```
project/
├── frontend/          # Astro SSR — Persian RTL UI
├── backend/
│   ├── main.py        # FastAPI app + routes
│   ├── agents/        # analysis_agent, rag_agent, facilitation
│   ├── services/      # sqlite, chroma, jira, embed, audio_transcribe,
│   │                  # gemini_errors, stats, ingest, facilitation
│   ├── models/        # Pydantic schemas
│   └── tests/         # unit + live tests
├── data/synthetic/    # sample meetings
├── data/meetings.db   # SQLite — summaries and tasks
├── data/chroma/       # ChromaDB persistence
├── docs/              # HTML docs + PDF + diagrams + CHANGELOG
└── scripts/           # run-dev, run-backend, run-frontend, tests, export-docs
```

13. Running the System

The full stack runs with one script (frees ports, starts both servers, cleans up on exit):

```
chmod +x scripts/run-dev.sh
./scripts/run-dev.sh
# Open: http://localhost:4321
```

Or run the two services separately:

```
./scripts/run-backend.sh    # FastAPI on :8000
./scripts/run-frontend.sh   # Astro on :4321
```

14. Tests

Type	Command	Count
Unit (mocked)	./scripts/run-tests.sh	150 pass (13 skip without optional/live credentials)
Live API & integration	./scripts/run-live-tests.sh	13 (real Gemini/Jira)
All	./scripts/run-all-tests.sh	Unit + live

Unit tests mock the LLM and external services for fast offline runs. The live suite exercises real FastAPI routes against real Google/Jira credentials, with a single Gemini end-to-end flow per session to avoid Pydantic AI event-loop issues.

15. Error Handling

Error	Behavior
Gemini timeout	One retry + Persian user message
Transcript without speaker	`[ناشناس]` label
Jira 401	Guide to check `.env`

Invalid / oversized audio	HTTP 400 (Persian) — format or `MAX_AUDIO_BYTES`
Gemini billing (403 dunning)	Persian billing guidance for transcription and analysis
RAG no match	"Not found in this meeting"

A dedicated `gemini_errors` service maps provider exceptions (`ModelHTTPError` and httpx errors) to user-friendly Persian `HTTPException`s, so a billing hold or invalid key never surfaces as a raw stack trace.

16. MLOps Observability (Pydantic Logfire)

To monitor agent and pipeline quality in real environments, we use Pydantic Logfire (OpenTelemetry). Module `backend/observability.py` at startup:

- `logfire.instrument_pydantic_ai()` — trace every agent run (analysis, RAG, facilitation)
- `logfire.instrument_httpx()` — Gemini and Jira requests
- `logfire.instrument_fastapi(app)` — API routes
- Manual spans on ingest: `ingest_meeting` → `analyze_transcript` → `index_vectors`

Variables `LOGFIRE_TOKEN`, `LOGFIRE_SERVICE_NAME`, `LOGFIRE_ENVIRONMENT` in `.env`. Without a token, no data is sent and overhead is negligible. `GET /api/health` returns `logfire: true/false`.

> **Upcoming sprints:** See Document 3 — multi-meeting RAG + full-text archive (Sprint 2), assignee mapping + Jira OAuth (Sprint 3), auth/workspace + calendar metadata (Sprint 4), PostgreSQL + queue + deployment (Sprint 5), and multi-agent capabilities (Sprint 6+). Live streaming STT is deliberately *not* on the roadmap — Gemini transcription covers MVP needs.